

1.MULTIPLE THREADS USING A SINGLE CLASS



SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

Multithreading Practice - CO3

QUESTIONS

Q1
Multiple Threads Using a Single Class
10 marks

Q2
Multiple thread classes
20 marks

Q3
Thread Synchronization
10 marks

3/3 answered

Q1 Multiple Threads Using a Single Class
10 marks

Topic: Thread class, multiple threads, if-else

Question:

Write a Java program using a single thread class to create three threads named Prime, Armstrong, and Reverse.

The program should contain a common run() method. Based on the thread name, use if-else statements to perform the following operations on a given number:

Prime → Check whether the number is prime.
Armstrong → Check whether the number is an Armstrong number.
Reverse → Find the reverse of the number.

Create and start all three threads from the main() method.

Input: 153

Expected Output:

Prime Thread: 153 is not a prime number
Armstrong Thread: 153 is an Armstrong number
Reverse Thread: Reverse of 153 = 351



SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

Your Code

```
1 import java.util.Scanner;
2 class Multithread extends Thread {
3     private int n;
4     Multithread(String name, int n) {
5         super(name);
6         this.n = n;
7     }
8     public void run() {
9         String name = getName();
10        if (name.equals("Prime")) {
11
12            if (isPrime(n)) {
13                System.out.println("Prime Thread: " + n + " is a prime number");
14            } else {
15                System.out.println("Prime Thread: " + n + " is not a prime number");
16            }
17        } else if (name.equals("Armstrong")) {
18
19            if (isArmstrong(n)) {
20                System.out.println("Armstrong Thread: " + n + " is an Armstrong number");
21            } else {
22                System.out.println("Armstrong Thread: " + n + " is not an Armstrong number");
23            }
24        } else if (name.equals("Reverse")) {
25            int rev = reverseNumber(n);
26            System.out.println("Reverse Thread: Reverse of " + n + " = " + rev);
27        }
```

Input

153

Output

Prime Thread: 153 is not a prime number
Armstrong Thread: 153 is an Armstrong number
Reverse Thread: Reverse of 153 = 351

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

10.00 / 10 marks

```

28     }
29 }
30 private boolean isPrime(int num) {
31     if (num <= 1) {
32         return false;
33     }
34     for (int i = 2; i * i <= num; i++) {
35         if (num % i == 0) {
36             return false;
37         }
38     }
39     return true;
40 }
41 private boolean isArmstrong(int num) {
42     int original = num;
43     int temp = num;
44     int digits = 0;
45     int sum = 0;
46
47     if (num == 0) {
48         return true;
49     }
50     while (temp > 0) {
51         digits++;
52         temp = temp / 10;
53     }
54     temp = num;
55     while (temp > 0) {
56         int digit = temp % 10;

```

```

54     temp = num;
55     while (temp > 0) {
56         int digit = temp % 10;
57         sum = sum + (int) Math.pow(digit, digits);
58         temp = temp / 10;
59     }
60     return sum == original;
61 }
62 private int reverseNumber(int num) {
63     int reverse = 0;
64     while (num > 0) {
65         int digit = num % 10;
66         reverse = reverse * 10 + digit;
67         num = num / 10;
68     }
69     return reverse;
70 }
71 public static void main(String[] args) throws Exception {
72     Scanner sc = new Scanner(System.in);
73     int n = sc.nextInt();
74     MultiThread prime = new MultiThread("Prime", n);
75     MultiThread armstrong = new MultiThread("Armstrong", n);
76     MultiThread reverse = new MultiThread("Reverse", n);
77     prime.start();
78     prime.join();
79     armstrong.start();
80     armstrong.join();
81     reverse.start();
82     reverse.join();
83     sc.close();

```

2.MULTIPLE THREAD CLASSES

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

Multithreading Practice - CO3

← Back

QUESTIONS

Q1 Multiple Threads Using a Single Class
10 marks

Q2 Multiple thread classes
20 marks

Q3 Thread Synchronization
10 marks

3/3 answered

Q2 Multiple thread classes 20 marks

Question:
Write a Java program to create three different thread classes to perform different operations on the same number.

Create the following classes:

PrimeThread → Check whether the given number is prime.
FactorialThread → Find the factorial of the given number.
ReverseThread → Find the reverse of the given number.

Each class should extend the Thread class and override the run() method.

Create and start all three threads from the main() method.

Input: 7

Expected Output:

Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7

Note: Since the threads execute concurrently, the order of output may vary.

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

Your Code

```
1 import java.util.Scanner;
2 class Main {
3     public static void main(String[] args) throws Exception {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         PrimeThread p = new PrimeThread(n);
7         FactorialThread f = new FactorialThread(n);
8         ReverseThread r = new ReverseThread(n);
9         p.start();
10        f.start();
11        r.start();
12        p.join();
13        f.join();
14        r.join();
15    }
16 }
17 class PrimeThread extends Thread {
18     int n;
19     PrimeThread(int n) {
20         this.n = n;
21     }
22     public void run() {
23         synchronized (Order.class) {
24             while (Order.turn != 1) {
25                 try {
26                     Order.class.wait();
27                 } catch (Exception e) {}
28             }

```

Input
7

Output
Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7

Visible Test Cases
Test Case 1 ✓
Test Case 2 ✓
Test Case 3 ✓
20.00 / 20 marks

```

24 while (Order.turn != 1) {
25     try {
26         Order.class.wait();
27     } catch (Exception e) {}
28 }
29 boolean prime = true;
30 if (n <= 1) {
31     prime = false;
32 } else {
33     for (int i = 2; i <= Math.sqrt(n); i++) {
34         if (n % i == 0) {
35             prime = false;
36             break;
37         }
38     }
39 }
40 if (prime)
41     System.out.println("Prime: " + n + " is a prime number");
42 else
43     System.out.println("Prime: " + n + " is not a prime number");
44 Order.turn = 2;
45 Order.class.notifyAll();
46 }
47 }
48 }
49 class FactorialThread extends Thread {
50     int n;
51     FactorialThread(int n) {
52         this.n = n;

```

```

49 class FactorialThread extends Thread {
50     int n;
51     FactorialThread(int n) {
52         this.n = n;
53     }
54     public void run() {
55         synchronized (Order.class) {
56             while (Order.turn != 2) {
57                 try {
58                     Order.class.wait();
59                 } catch (Exception e) {}
60             }
61             long fact = 1;
62             for (int i = 1; i <= n; i++) {
63                 fact = fact * i;
64             }
65             System.out.println("Factorial: " + n + "! = " + fact);
66             Order.turn = 3;
67             Order.class.notifyAll();
68         }
69     }
70 }
71 class ReverseThread extends Thread {
72     int n;
73     ReverseThread(int n) {
74         this.n = n;
75     }
76     public void run() {
77         synchronized (Order.class) {

```

```
69     }
70 }
71 class ReverseThread extends Thread {
72     int n;
73     ReverseThread(int n) {
74         this.n = n;
75     }
76     public void run() {
77         synchronized (Order.class) {
78             while (Order.turn != 3) {
79                 try {
80                     Order.class.wait();
81                 } catch (Exception e) {}
82             }
83             int temp = n;
84             int reverse = 0;
85             while (temp != 0) {
86                 reverse = reverse * 10 + temp % 10;
87                 temp = temp / 10;
88             }
89             System.out.println("Reverse: Reverse of " + n + " = " + reverse);
90             Order.turn = 4;
91             Order.class.notifyAll();
92         }
93     }
94 }
95 class Order {
96     static int turn = 1;
97 }
```

3.THREAD SYNCHRONIZATION

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

QUESTIONS

Q1
Multiple Threads Using a Single Class
10 marks

Q2
Multiple thread classes
20 marks

Q3
Thread Synchronization
10 marks

3/3 answered

Q3 Thread Synchronization 10 marks

opic: synchronized, shared resource, race condition

Question:

Write a Java program to demonstrate thread synchronization using a shared Counter object.

Create a class Counter containing an integer variable count, initially set to 0.

Create two threads, ThreadA and ThreadB. Both threads should increment the same count variable 1000 times.

Implement the increment() method using the synchronized keyword so that only one thread can modify the counter at a time.

After both threads complete their execution, display the final value of count.

Expected Output:

ThreadA completed
ThreadB completed
Final Count = 2000

Requirements:

Use a shared Counter object.
Both threads must access the same Counter object.
Use synchronized for the increment operation.
Use join() to ensure both threads finish before displaying the final count.

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

 SANGEPU VENKATA PRAVEEN
192373013RD

Your Code

```
1 class Main {
2     public static void main(String[] args) throws Exception {
3         Counter co = new Counter();
4         ThreadA T1 = new ThreadA(co);
5         ThreadB T2 = new ThreadB(co);
6         T1.start();
7         T2.start();
8         T1.join();
9         T2.join();
10        System.out.println("Final Count = " + co.count);
11    }
12 }
13 class Counter {
14     int count = 0;
15     public synchronized void counterIncrement() {
16         count++;
17     }
18 }
19 class ThreadA extends Thread {
20     Counter c;
21     ThreadA(Counter co) {
22         c = co;
23     }
24     public void run() {
25         for (int i = 0; i < 1000; i++) {
26             c.counterIncrement();
27         }
28     }
29 }
```

Input

stdin input

Output

ThreadA completed
ThreadB completed
Final Count = 2000

Visible Test Cases

Test Case 1 ✓
Test Case 2 ✓
Test Case 3 ✓

10.00 / 10 marks

10.00 / 10 marks

```
20 Counter c;  
21 ThreadA(Counter co) {  
22     c = co;  
23 }  
24 public void run() {  
25     for (int i = 0; i < 1000; i++) {  
26         c.counterIncrement();  
27     }  
28     System.out.println("ThreadA completed");  
29 }  
30 }  
31 class ThreadB extends Thread {  
32     Counter c;  
33     ThreadB(Counter co) {  
34         c = co;  
35     }  
36     public void run() {  
37         for (int i = 0; i < 1000; i++) {  
38             c.counterIncrement();  
39         }  
40         System.out.println("ThreadB completed");  
41     }  
42 }
```

 Submitted